Diraschool

API Documentation

Version:	v1.0
Production:	https://diraschool-api-production.up.railway.app
Local:	http://localhost:3000
Prefix:	/api/v1
Date:	2026-04-12

Table of Contents

1. Overview
2. Authentication
3. Response Format
4. Error Codes
5. Pagination
6. Rate Limiting
7. Roles & Permissions
8. Enumerations
9. Endpoints
    - Health Check
    - Auth
    - Users (Staff)
    - Schools
    - Classes
    - Students
    - Subjects
    - Attendance
    - Exams
    - Results
    - Fees
    - Report Cards
    - Timetable
    - Library
    - Transport
    - Settings
    - Audit Logs
    - Parent Portal
10. Data Models Reference

DiraSchool API Reference

Base URL: `https://api.diraschool.com/api/v1`

Version: v1

Updated: May 2026

Overview

DiraSchool is a multi-tenant SaaS platform for Kenyan CBC schools. Every API request operates within a single school's data scope — no request can read or modify another school's data.

Authentication

All protected endpoints require a valid session cookie set by `POST /auth/login`. The cookie is HTTP-only and cannot be read by JavaScript. Pass it automatically via the browser or include it explicitly in server-to-server requests.

```
Cookie: token=<jwt>
```

Rate limits apply globally (200 req/IP/min) and more strictly on auth endpoints (20 req/IP/15 min).

Response Format

All responses follow a consistent envelope:

```
{ "success": true, "data": { ... } }
{ "success": false, "message": "Human-readable error" }
```

HTTP status codes are conventional: `200` success, `201` created, `400` bad request, `401` unauthenticated, `403` forbidden, `404` not found, `429` rate limited, `500` server error.

Roles

Role	Identifier
Superadmin	`SUPERADMIN`
School Admin	`SCHOOL_ADMIN`
Director	`DIRECTOR`
Head Teacher	`HEAD_TEACHER`
Deputy Head Teacher	`DEPUTY_HEAD_TEACHER`
Teacher	`TEACHER`
Department Head	`DEPARTMENT_HEAD`
Secretary	`SECRETARY`
Accountant	`ACCOUNTANT`
Parent	`PARENT`

Health

GET `/health`

No authentication required. Returns platform status.

Response

```
{
  "status": "ok",
  "timestamp": "2026-05-07T10:00:00.000Z",
  "uptime": 86400,
  "services": {
    "api": "up",
    "mongodb": "up",
    "redis": "up",
    "storage": "configured"
  },
  "performance": { "avgResponseMs": 42, "samples": 1200 },
  "memory": { "heapUsedMb": 128, "heapTotalMb": 256, "rssMb": 180 }
}
```


Authentication `/api/v1/auth`

Rate-limited to **20 requests / IP / 15 minutes**.

POST `/auth/register`

Register a new school and create the first admin account. Starts a 30-day free trial.

Body

```
{
  "schoolName": "Sunrise Academy",
  "county": "Nairobi",
  "address": "Westlands",
  "phone": "0712345678",
  "principalName": "Jane Mwangi",
  "firstName": "Jane",
  "lastName": "Mwangi",
  "email": "jane@sunriseacademy.ke",
  "password": "SecurePass123!"
}
```

Response `201` — school and user created, verification email sent.

POST `/auth/login`

Authenticate and receive a session cookie.

Body

```
{ "email": "jane@sunriseacademy.ke", "password": "SecurePass123!" }
```

Response `200` — sets HTTP-only `token` cookie. Returns user profile and school context.

POST /auth/forgot-password

Send a password-reset link to the user's email.

Body `{ "email": "jane@sunriseacademy.ke" }`

POST /auth/reset-password/:token

Set a new password using the token from the reset email.

Body `{ "password": "NewSecurePass456!" }`

POST /auth/accept-invite/:token

Staff first-time login — set own password using the invite token from email.

Body `{ "password": "MyFirstPassword!" }`

POST /auth/verify-email

Trigger email verification (sends OTP code and one-click link).

GET /auth/verify-email/:token

One-click email verification via link token.

POST /auth/resend-verification

Resend the verification email.

POST /auth/logout

Clear session cookie.

GET /auth/me

Get current user profile with school context.

Response

```
{
  "user": {
    "_id": "...",
    "firstName": "Jane",
    "lastName": "Mwangi",
    "email": "jane@sunriseacademy.ke",
    "role": "SCHOOL_ADMIN",
    "schoolId": "...",
    "emailVerified": true,
    "phoneVerified": false
  },
  "school": { "name": "Sunrise Academy", "county": "Nairobi" }
}
```

PATCH `/auth/me`

Update own profile. Accepted fields: `firstName`, `lastName`, `email`, `phone`.

POST `/auth/change-password`

Change own password. Accessible even when `mustChangePassword` is `true`.

Body `{ "currentPassword": "...", "newPassword": "..." }`

Users `/api/v1/users`

School admin manages staff accounts. All routes require admin-level role.

GET `/users`

List all users in the school.

Query params: `page`, `limit`, `role`, `isActive`, `search`

POST `/users`

Create a new staff account. Sends an invite email; the user sets their own password via the invite link.

Body

```
{
  "firstName": "Peter",
  "lastName": "Odhiambo",
  "email": "peter@sunriseacademy.ke",
  "role": "TEACHER",
  "phone": "0722000000"
}
```

GET `/users/:id`

Get staff member details.

PATCH `/users/:id`

Update staff details: name, role, phone, `isActive`.

DELETE `/users/:id`

Delete staff account.

POST `/users/:id/resend-invite`

Re-send invitation email.

POST `/users/:id/reset-password`

Admin-forced password reset — sets `mustChangePassword = true`, user must set a new password on next login.

Classes `/api/v1/classes`

GET `/classes/my-class`

Get the class where the authenticated teacher is assigned as class teacher.

GET `/classes`

List all classes. Teachers see only their own class.

Query params: `page`, `limit`, `level`

CBC levels: `Lower Primary` | `Upper Primary` | `JSS`

GET `/classes/:id`

Class details including assigned teacher and student count.

POST `/classes`

Body

```
{
  "name": "Grade 4",
  "stream": "A",
  "level": "Upper Primary",
  "classTeacherId": "..."
}
```

PATCH `/classes/:id`

Update class name, stream, level, or teacher assignment.

DELETE `/classes/:id`

Soft delete — historical records are preserved.

POST `/classes/:id/promote`

Bulk promote all students in the class to the next CBC level for a new academic year.

Body `{ "targetClassId": "..." }`

Students `/api/v1/students`

GET `/students`

List all students. Filterable by class, status, search term.

Query params: `classId`, `status` (`active` | `withdrawn` | `transferred`), `search`, `page`, `limit`

GET `/students/:id`

Full student profile: demographics, guardian details, class, photo URL, status.

POST `/students`

Enroll a new student.

Body

```
{
  "firstName": "Amara",
  "lastName": "Kipchoge",
  "admissionNumber": "SUN/2026/001",
  "dateOfBirth": "2015-03-12",
  "gender": "Female",
  "classId": "...",
  "guardianName": "David Kipchoge",
  "guardianPhone": "0733000000",
  "guardianRelation": "Father"
}
```

POST `/students/import`

Bulk enroll students from a CSV file (feature-gated by subscription plan). Returns an async `jobId`.

Form: `multipart/form-data` — field `file` (CSV).

GET `/students/import/:jobId/status`

Poll CSV import job status.

Response `{ "status": "processing" | "complete" | "failed", "imported": 42, "errors": [] }`

PATCH `/students/:id`

Update student details, class assignment, or guardian info.

POST `/students/:id/photo`

Upload student profile photo. `multipart/form-data`, field `photo`.

POST `/students/:id/transfer`

Transfer student to a different class. Transfer history is preserved.

Body `{ "targetClassId": "..." }`

POST `/students/:id/withdraw`

Mark student as withdrawn. Historical records are retained.

Attendance `/api/v1/attendance`

GET `/attendance/registers`

List attendance registers. Filterable by class, date range, status.

Query params: `classId`, `from`, `to`, `status` (`draft` | `submitted`), `page`, `limit`

POST `/attendance/registers`

Create a daily register for a class. Auto-populates with all active students.

Body `{ "classId": "...", "date": "2026-05-07" }`

GET `/attendance/registers/:id`

Register detail with per-student attendance status.

PATCH `/attendance/registers/:id`

Update attendance status per student.

Body

```
{
  "records": [
    { "studentId": "...", "status": "Present" },
    { "studentId": "...", "status": "Absent" }
  ]
}
```

Status values: `Present` | `Absent` | `Late` | `Excused`

POST `/attendance/registers/:id/submit`

Submit and lock the register. Locked registers cannot be edited.

Subjects `/api/v1/subjects`

GET `/subjects/my-subjects`

Get subjects assigned to the authenticated teacher.

GET `/subjects`

List all subjects. Teachers see only their assigned subjects.

GET `/subjects/:id`

Subject detail with assigned teachers and HOD.

POST `/subjects`

Body `{ "name": "Mathematics", "code": "MTH", "description": "..." }`

PATCH `/subjects/:id`

Update subject details.

DELETE `/subjects/:id`

Delete subject.

PATCH `/subjects/:id/teachers`

Assign teachers and HOD to a subject.

Body `{ "teacherIds": ["..."], "hodId": "..." }`

PATCH `/subjects/:id/self-assign`

Toggle own assignment to a subject.

Exams `/api/v1/exams`

GET `/exams`

List exams. Filter by class, subject, term, year.

Query params: `classId`, `subjectId`, `term`, `academicYear`, `page`, `limit`

POST `/exams`

Create an exam.

Body

```
{
  "name": "Term 2 Midterm",
  "type": "Midterm",
  "subjectId": "...",
  "classId": "...",
  "totalMarks": 100,
  "date": "2026-06-15",
  "term": "Term 2",
  "academicYear": "2026"
}
```

Exam types: `Opener` | `Midterm` | `Endterm` | `SBA`

GET `/exams/:id`

Exam detail.

PATCH `/exams/:id`

Update exam.

DELETE `/exams/:id`

Delete exam.

Results `/api/v1/results`

CBC grades are calculated automatically from marks.

Primary scale: EE (≥75%) → ME (50–74%) → AE (25–49%) → BE (<25%)

JSS 8-point scale: EE1 → EE2 → ME1 → ME2 → AE1 → AE2 → BE1 → BE2

POST `/results/bulk`

Bulk upsert marks for all students in one exam.

Body

```
{
  "examId": "...",
  "results": [
    { "studentId": "...", "marksObtained": 78 },
    { "studentId": "...", "marksObtained": 45 }
  ]
}
```

Response includes the calculated CBC grade for each student.

GET `/results`

List results. Filter by exam, student, class.

GET `/results/:id`

Single result record.

PATCH `/results/:id`

Update a single result entry.

Fees `/api/v1/fees`

Fee Structures

`GET /FEES/STRUCTURES` ■

List fee structures. Filter by term and year.

Query params: `term`, `academicYear`, `classId`

`POST /FEES/STRUCTURES` ■ `ADMIN`

Create a fee structure with line items.

Body

```
{
  "name": "Term 2 2026",
  "term": "Term 2",
  "academicYear": "2026",
  "classIds": ["..."],
  "lineItems": [
    { "name": "Tuition", "amount": 15000 },
    { "name": "Activity", "amount": 2000 },
    { "name": "Lunch", "amount": 5000 }
  ]
}
```

`POST /FEES/STRUCTURES/ADAPT` ■ `ADMIN`

Copy an existing structure to a new term/year.

Body `{ "sourceStructureId": "...", "term": "Term 3", "academicYear": "2026" }`

`GET /FEES/STRUCTURES/:ID` ■

Structure detail with all line items.

`PATCH /FEES/STRUCTURES/:ID` ■ `ADMIN`

Update structure.

`DELETE /FEES/STRUCTURES/:ID` ■ `ADMIN`

Delete structure.

Payments

`GET /FEES/PAYMENTS` ■

List payments. Filter by student, method, date range, status.

Query params: `studentId`, `method`, `from`, `to`, `status`, `page`, `limit`

`POST /FEES/PAYMENTS` ■ `ACCOUNTANT | SECRETARY`

Record a fee payment.

Body

```
{
  "studentId": "...",
  "feeStructureId": "...",
  "amount": 15000,
  "method": "M-Pesa",
  "reference": "QAB12345",
  "paymentDate": "2026-05-07"
}
```

Payment methods: `cash` | `M-Pesa` | `bank transfer` | `cheque`

`GET /FEES/PAYMENTS/:ID` ■

Payment receipt details.

`POST /FEES/PAYMENTS/:ID/REVERSE` ■ `ACCOUNTANT`

Reverse a payment. Restores fee balance.

Body `{ "reason": "Duplicate entry" }`

`POST /FEES/PAYMENTS/:ID/ISSUE-RECEIPT` ■

Generate and send receipt to parent.

Other Fee Endpoints

Endpoint	Description
`GET /fees/balance?studentId=...`	Student fee balance (owed vs. paid)
`GET /fees/bulk-stats`	Aggregate stats (total collected, outstanding)
`GET /fees/dashboard-summary`	Financial dashboard KPIs

Report Cards `/api/v1/report-cards`

Report cards are generated from exam results entered via `/results/bulk`.

GET `/report-cards`

List report cards. Filter by student, class, term, year.

GET `/report-cards/:id`

Full report card: CBC grades per subject, weighted averages, attendance summary, remarks, signature lines.

PATCH `/report-cards/:id/remarks`

Update class teacher remarks.

Body `{ "classTeacherRemark": "An excellent student who shows great initiative." }`

PATCH `/report-cards/:id/subjects/:subjectId/remark`

Update subject-level teacher remark.

Body `{ "remark": "Shows strong numeracy skills." }`

GET `/report-cards/annual-summary`

Year-end summary across all students and classes.

Lesson Plans `/api/v1/lesson-plans`

GET `/lesson-plans`

List lesson plans. Filter by teacher, subject, date range.

POST `/lesson-plans`

Upload a lesson plan with optional images (max 20 images, 10 MB each).

Form: `multipart/form-data` — fields: `subjectId`, `classId`, `date`, `title`, `content`, `images[]`

GET `/lesson-plans/:id`

Lesson plan detail.

DELETE `/lesson-plans/:id`

Delete own lesson plan.

POST `/lesson-plans/:id/share`

Share with another teacher.

Body `{ "teacherId": "..." }`

DELETE `/lesson-plans/:id/share/:teacherId`

Unshare with a teacher.

Schools `/api/v1/schools`

School-Admin (Own School)

`GET /SCHOOLS/ME` ■

Own school profile: name, principal, address, academic year, term dates, logo URL.

`PATCH /SCHOOLS/ME` ■ `PRINCIPAL | DIRECTOR | ADMIN`

Update school profile.

`POST /SCHOOLS/ME/SMS-SENDER-ID-REQUEST` ■

Request a custom SMS sender ID. Subject to approval by DiraSchool superadmin.

Body `{ "requestedSenderId": "SUNRISE" }`

Superadmin (All Schools)

`GET /SCHOOLS` ■ `SUPERADMIN`

List all schools with student/staff counts.

`POST /SCHOOLS` ■ `SUPERADMIN`

Create a new school tenant.

`GET /SCHOOLS/:ID` ■ `SUPERADMIN`

School detail with staff breakdown by role.

`PATCH /SCHOOLS/:ID` ■ `SUPERADMIN`

Update school info, status, subscription tier, plan limits.

`PATCH /SCHOOLS/:ID/SUBSCRIPTION` ■ `SUPERADMIN`

Update subscription plan and trial status.

Parent Portal `/api/v1/parent`

Feature-gated (plan-tier). Accessible only to users with role `PARENT`. Strictly scoped to their own children.

GET `/parent/children`

List all children enrolled in the school linked to this parent account.

GET `/parent/children/:studentId/fees`

Child's fee balance and payment history.

GET `/parent/children/:studentId/attendance`

Child's attendance record.

GET `/parent/children/:studentId/results`

Child's exam results and report cards.

Audit Logs `/api/v1/audit-logs`

Feature-gated. Accessible to admin-level roles only.

GET `/audit-logs`

List audit logs.

Query params: `userId`, `action`, `resource`, `from`, `to`, `page`, `limit`

Every log entry: `{ userId, userRole, action, resource, resourceId, timestamp, changes, schoolId }`

Settings `/api/v1/settings`

GET `/settings`

School settings: calendar, holidays, working days, geofence, check-in times, logo.

PUT `/settings`

Update settings.

POST `/settings/logo`

Upload school logo. `multipart/form-data`, field `logo`.

POST `/settings/holidays`

Add a school holiday.

Body `{ "name": "Mashujaa Day", "date": "2026-10-20" }`

DELETE `/settings/holidays/:holidayId`

Remove holiday.

PUT `/settings/geofence`

Set geofence coordinates and radius for check-in validation.

Body `{ "latitude": -1.286389, "longitude": 36.817223, "radiusMetres": 200 }`

PUT `/settings/checkin-times`

Set check-in/check-out deadline times.

Body `{ "checkInDeadline": "07:30", "checkOutTime": "17:00" }`

Timetable `/api/v1/timetables`

Feature-gated.

GET `/timetables`

List timetables. Filter by class, term, year.

GET `/timetables/:id`

Timetable detail with all time slots (subject, teacher, room).

POST `/timetables`

Create timetable for a class.

Body `{ "classId": "...", "term": "Term 2", "academicYear": "2026" }`

PUT `/timetables/:id/slots`

Update time slots.

Body

```
{
  "slots": [
    { "day": "Monday", "period": 1, "subjectId": "...", "teacherId": "...", "room": "Room 4" }
  ]
}
```

DELETE `/timetables/:id`

Delete timetable.

Transport `/api/v1/transport`

Feature-gated.

GET `/transport/routes`

List transport routes.

GET `/transport/routes/:id`

Route detail: driver, vehicle, assigned students.

POST `/transport/routes`

Create route.

Body `{ "name": "Route A - Westlands", "driverName": "...", "vehicle": "KAA 000A", "departureTime": "06:30" }`

PATCH `/transport/routes/:id`

Update route.

DELETE `/transport/routes/:id`

Delete route.

POST `/transport/routes/:id/assign`

Assign students to route.

Body `{ "studentIds": ["..."] }`

POST `/transport/routes/:id/unassign`

Remove students from route.

Body `{ "studentIds": ["..."] }`

Dashboard `/api/v1/dashboard`

GET `/dashboard`

School-wide KPIs: student count, fees collected this term, attendance rate, recent activity.

GET `/dashboard/teacher`

Teacher KPIs: assigned classes, student count, upcoming exams, attendance summary.

Notifications `/api/v1/notifications`

GET `/notifications`

List notifications for current user with pagination.

GET `/notifications/unread-count`

Get count of unread notifications.

POST `/notifications/mark-all-read`

Mark all notifications as read.

POST `/notifications/:id/read`

Mark single notification as read.

Email Events `/api/v1/email`

GET `/email/events`

List email delivery events: sent / opened / bounced.

GET `/email/events/:id`

Email event detail.

Pricing `/api/v1/pricing`

GET `/pricing/calculate`

No auth required. Calculate subscription cost.

Query params: `students` (integer), `billing` (`per-term` | `annual` | `multi-year`)

Response

```
{
  "students": 200,
  "billing": "annual",
  "subtotal": 23000,
  "total": 62100,
  "costPerStudent": 115,
  "multiplier": 2.70
}
```

Pricing formula: **KES 12,000 base + KES 55 × students** per term.

Annual (3 terms, 2 billed): 10% discount (×2.70).

Multi-year (3 terms upfront): 15% discount (×2.55).

Export `/api/v1/export`

All endpoints return `text/csv`.

GET `/export/students`

Export all students to CSV.

GET `/export/payments`

Export all payments to CSV.

GET `/export/staff`

Export all staff to CSV.

SMS `/api/v1/sms`

POST `/sms/send`

Send SMS to a single parent/guardian.

Body `{ "phone": "+254712345678", "message": "Fee balance reminder..." }`

POST `/sms/broadcast`

Broadcast SMS to multiple recipients.

Body

```
{
  "recipients": "class" | "parents" | "staff",
  "classId": "...",
  "message": "School will be closed on Friday."
}
```

SMS cap: **5 messages per parent per term** included in subscription. Additional messages require purchased credits.

GET `/sms/history`

SMS send history with delivery status.

Query params: `from`, `to`, `trigger`, `page`, `limit`

GET `/sms/deliveries`

Per-recipient delivery reports.

GET `/sms/stats`

SMS statistics: sent, delivered, failed, cap usage, credit balance.

GET `/sms/credit-packs`

List available SMS credit packs for purchase.

Response

```
[
  { "id": "sms_200",  "credits": 200,  "amountKes": 300,  "label": "200 SMS" },
  { "id": "sms_500",  "credits": 500,  "amountKes": 700,  "label": "500 SMS" },
  { "id": "sms_1000", "credits": 1000, "amountKes": 1200, "label": "1,000 SMS" },
  { "id": "sms_2500", "credits": 2500, "amountKes": 2750, "label": "2,500 SMS" }
]
```

POST `/sms/credit-packs/checkout`

Initiate Paystack checkout to purchase SMS credits.

Body `{ "packId": "sms_500" }`

Response `{ "checkoutUrl": "https://checkout.paystack.com/..." }`

Inbound / Webhooks (Public)

Method	Endpoint	Description
POST	`/sms/inbound`	Africa's Talking inbound SMS callback
POST	`/sms/dlr`	Africa's Talking delivery report (DLR) callback

OTP `/api/v1/otp`

Phone number verification for parent accounts via SMS.

POST `/otp/send`

Send a 6-digit OTP to the parent's phone number.

Body `{ "phone": "+254712345678" }`

Rate limit: 3 sends per phone per 10 minutes. OTP expires in 10 minutes.

POST `/otp/verify`

Verify the OTP and mark `phoneVerified = true` on the user account.

Body `{ "phone": "+254712345678", "otp": "482910" }`

Rate limit: 5 verify attempts before OTP is invalidated (request a new one).

M-Pesa `/api/v1/mpesa`

Safaricom Daraja C2B integration. Callback endpoints are IP-whitelisted to Safaricom production and sandbox ranges.

GET `/mpesa/settings`

Get M-Pesa configuration for the school.

PUT `/mpesa/settings`

Update M-Pesa credentials.

Body

```
{
  "paybillNumber": "247247",
  "accountPrefix": "STU",
  "consumerKey": "...",
  "consumerSecret": "..."
}
```

POST `/mpesa/register-c2b/:schoolId`

Register school paybill for C2B callbacks with Safaricom.

GET `/mpesa/payments`

List M-Pesa payments.

GET `/mpesa/payments/unallocated`

List unallocated M-Pesa payments (received but not yet matched to a student).

POST `/mpesa/payments/allocate`

Allocate an unmatched payment to a student.

Body `{ "paymentId": "...", "studentId": "..." }`

POST `/mpesa/payments/manual`

Manually record an M-Pesa payment.

GET `/mpesa/payments/summary`

M-Pesa payment summary for the current term.

GET `/mpesa/payments/student/:studentId`

All M-Pesa payments for a student.

M-Pesa Callbacks (Public, IP-Protected)

Method	Endpoint	Description
POST	`/mpesa/validation`	Safaricom C2B validation callback
POST	`/mpesa/confirmation`	Safaricom C2B confirmation callback

Visitors `/api/v1/visitors`

GET `/visitors`

List visitor logs. Filter by date range, status.

POST `/visitors`

Register a visitor.

Body

```
{
  "name": "John Kamau",
  "company": "Ministry of Education",
  "nationalId": "12345678",
  "purpose": "School inspection",
  "date": "2026-05-07"
}
```

PATCH `/visitors/:id`

Update visitor record (check-out time, notes).

DELETE `/visitors/:id`

Delete visitor record.

Check-ins `/api/v1/checkins`

Staff geofence-based or manual check-in.

POST `/checkins`

Create a check-in record.

Body `{ "method": "geofence" | "manual", "latitude": -1.286, "longitude": 36.817 }`

GET `/checkins/today`

Own check-ins for today.

GET `/checkins/roster`

Daily roster of all staff check-ins.

GET `/checkins/staff/:staffId`

Check-in history for a specific staff member.

Leave `/api/v1/leave`

POST `/leave`

Apply for leave.

Body

```
{
  "type": "Annual" | "Sick" | "Maternity" | "Paternity" | "Compassionate" | "Study",
  "startDate": "2026-05-20",
  "endDate": "2026-05-22",
  "reason": "Family emergency"
}
```

GET `/leave`

List own leave applications.

GET `/leave/balances`

Available leave balance by type.

GET `/leave/pending-count`

Count of pending leave requests awaiting approval.

GET `/leave/on-leave-today`

Staff currently on approved leave today.

GET `/leave/summary`

Leave summary across all staff.

GET `/leave/:id`

Leave application detail.

PATCH `/leave/:id/approve`

Approve leave request.

PATCH `/leave/:id/reject`

Reject with reason.

Body `{ "reason": "Insufficient cover" }`

DELETE `/leave/:id`

Cancel own leave application (if not yet approved).

Payroll `/api/v1/payroll`

Salary Grades

Method	Endpoint	Description
GET	`/payroll/grades`	List salary grades
POST	`/payroll/grades`	Create grade
PATCH	`/payroll/grades/:id`	Update grade
DELETE	`/payroll/grades/:id`	Delete grade

Body (create):

```
{
  "name": "Teacher Grade B5",
  "baseSalary": 45000,
  "allowances": [
    { "name": "House", "amount": 8000 },
    { "name": "Transport", "amount": 3000 }
  ]
}
```

Payroll Runs

Method	Endpoint	Description
GET	`/payroll/runs`	List payroll runs
GET	`/payroll/runs/:id`	Run detail (all staff + calculated salaries)
POST	`/payroll/runs`	Generate new run
DELETE	`/payroll/runs/:id`	Delete unapproved run
POST	`/payroll/runs/:id/approve`	Approve run (locks from further editing)
POST	`/payroll/runs/:id/paid`	Mark as paid

Body (create run):

```
{ "period": "May 2026", "type": "monthly" | "termly" }
```

Onboarding `/api/v1/onboarding`

GET `/onboarding/status`

Get onboarding checklist status.

Response

```
{
  "steps": {
    "schoolProfile": true,
    "staffAdded": true,
    "studentsAdded": false,
    "classesCreated": true,
    "subjectsAdded": true,
    "feeStructureCreated": false
  },
  "percentComplete": 66
}
```

POST `/onboarding/complete`

Mark onboarding as complete.

Subscriptions `/api/v1/subscriptions`

POST `/subscriptions/paystack/checkout`

Initiate Paystack checkout for subscription renewal.

Body `{ "plan": "annual", "students": 200 }`

Response `{ "checkoutUrl": "https://checkout.paystack.com/..." }`

GET `/subscriptions/paystack/status/:merchantReference`

Get payment status.

GET `/subscriptions/payments`

List subscription payment history.

POST `/subscriptions/paystack/webhook`

Public. HMAC-SHA512 verified Paystack webhook. Handles both subscription renewals (`metadata.type = 'subscription'`) and SMS credit top-ups (`metadata.type = 'sms_credits'`).

Admin Portal `/api/v1/admin`

All routes require `SUPERADMIN` role.

GET `/admin/stats`

Platform-wide statistics: total schools, students, staff.

GET `/admin/schools`

List all schools with statuses and subscription info.

GET `/admin/schools/:id`

School detail with staff breakdown.

PATCH `/admin/schools/:id/status`

Update school status.

Body `{ "status": "active" | "suspended" | "trial" }`

PATCH `/admin/schools/:id/sms-sender-id`

Approve a school's SMS sender ID request.

Body `{ "approved": true, "senderId": "SUNRISE" }`

GET `/admin/audit-logs`

System-wide audit logs across all schools.

GET `/admin/users`

List all superadmin users.

PATCH `/admin/users/:id/toggle`

Activate or deactivate a superadmin user.

POST `/admin/monitoring-test`

Trigger a monitoring health check.

GET `/admin/sms-analytics`

SMS delivery analytics across all schools: delivery rate, success rate, credit consumption per school.

Query params: `from`, `to`, `term`, `academicYear`

Environment Variables

Required

Variable	Purpose
`MONGO_URI`	MongoDB connection string
`JWT_SECRET`	Session token signing key (min 32 chars)
`CLIENT_URL`	Web frontend domain (used in email links)
`REDIS_URL`	Redis connection for BullMQ queue
`ZEPTOMAIL_API_KEY`	Transactional email via ZeptoMail (Zoho)

Optional / Feature-Gated

Variable	Default	Purpose
`PORT`	`3000`	API server port
`NODE_ENV`	`development`	Environment flag
`JWT_EXPIRES_IN`	`1d`	Session token TTL
`AT_USERNAME`	—	Africa's Talking username (SMS)
`AT_API_KEY`	—	Africa's Talking API key
`AT_SENDER_ID`	—	SMS sender ID (per school, after approval)
`AT_TEST_NUMBERS`	—	Redirect SMS to test numbers in dev/QA
`DO_SPACES_KEY`	—	DigitalOcean Spaces API key
`DO_SPACES_SECRET`	—	DO Spaces secret
`DO_SPACES_BUCKET`	—	DO Spaces bucket name
`DO_SPACES_REGION`	`ams3`	DO Spaces region
`SENTRY_DSN`	—	Sentry error monitoring
`PAYSTACK_ENABLED`	`false`	Enable Paystack payments
`PAYSTACK_SECRET_KEY`	—	Required if Paystack enabled
`MPESA_CONSUMER_KEY`	—	Safaricom Daraja consumer key
`MPESA_CONSUMER_SECRET`	—	Daraja secret
`MPESA_PASSKEY`	—	M-Pesa passkey
`MPESA_SHORTCODE`	—	School M-Pesa shortcode
`MPESA_ENV`	`production`	`production` or `sandbox`
`MPESA_CALLBACK_BASE_URL`	—	Public URL for Safaricom callbacks

Error Reference

Code	Meaning
400	Bad request — validation failed
401	Unauthenticated — no valid session cookie
403	Forbidden — authenticated but insufficient role
404	Resource not found

Code	Meaning
429	Rate limit exceeded
502	Upstream provider error (SMS, email)
503	Service temporarily unavailable (Redis down)

DiraSchool API — Dentrix Technologies, Kenya. Integration support: contact@diraschool.com